



LECTURE 4

DEEP REPRESENTATION LEARNING I

What is Learning?

PRESS **→** OR SPACE. 

© Marc Rußwurm

Licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#).

You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.

Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

Two Questions for Lectures 3–4

LECTURE 3

How does a neural network learn?

LECTURE 4

**Why should what it learns work
beyond the training data?**

Learning Outcomes Roadmap

Learning outcome	Block
☐ Explain why representation choice determines task difficulty.	Why Learn Representations?
☐ Interpret logistic regression as a linear decision boundary in feature space.	Why Learn Representations?
☐ Explain how an MLP learns transformations of feature space.	Multilayer Perceptrons
☐ Distinguish representation learner vs final decision layer in a network.	Multilayer Perceptrons
☐ Explain the roles of loss, gradient descent, and backpropagation.	Gradient Descent & Backpropagation

Today: Lecture Structure

Three major blocks guide the narrative.

1

Why Learn Representations?

2

Multilayer Perceptrons

3

**Gradient Descent &
Backpropagation**

Start with representation quality and geometric separability.

Good representations make hard problems easy.

Why Learn Representations?

What is a representation?

- Same information can be represented in more or less useful ways.
- Utility depends on whether structure is exposed for the task.
- Learning often succeeds or fails because of representation choice.

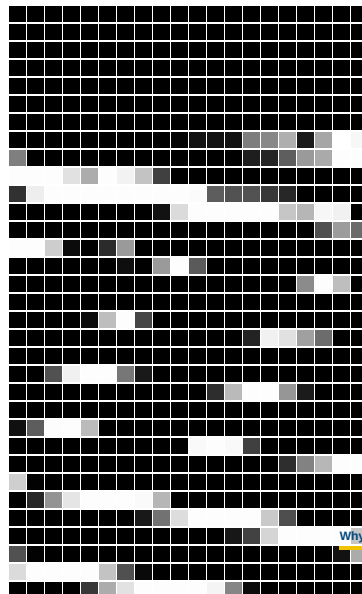
784 numbers. One digit.

Can you find a representation that reveals the structure?

Columns: 14–56



20 × 40 — Same 784 values. Different arrangement.



Same data. Different representation.

$$[x \in \mathbb{R}^{784}] \quad \longleftrightarrow \quad X \in \mathbb{R}^{28 \times 28}$$

No information was added.

The representation exposed spatial structure.

An MLP can still process the same image as a 784-dimensional vector.

Bridge from Lecture 3

Lecture 3 workflow:

data → manually designed representation → model → prediction

Students improved performance by:

- choosing features,
- engineering NDVI,
- adding complementary variables.

In Lecture 3, we designed the representation ourselves.

Logistic Regression in Feature Space

$$x \mapsto w^{\top} x + b \mapsto \sigma(\cdot) \mapsto \hat{y}$$

Samples are points in feature space.

Coordinates are feature values.

Logistic regression learns a **linear decision boundary**.

In 2D: line. In higher dimensions: hyperplane.

Representation Determines Difficulty

POOR REPRESENTATION

Classes overlap in awkward geometry.

Simple linear boundary struggles.

USEFUL REPRESENTATION

Classes become easier to separate.

Simple linear classifier can be sufficient.

What if we learn the representation?

Lecture 3

$\mathbf{x}$ → feature engineering → logistic regression

Lecture 4

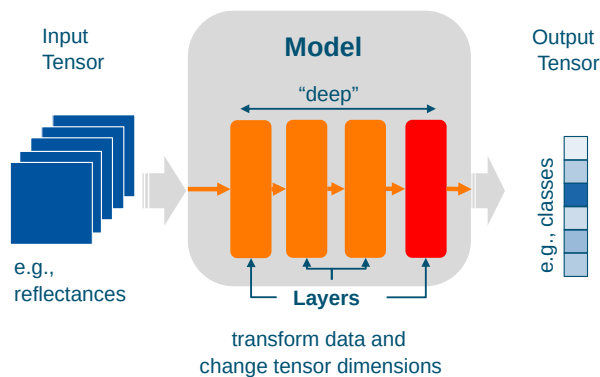
$\mathbf{x}$ → learned representation → decision layer

Next block: Multilayer Perceptrons.

Two Core Topics in This Lecture

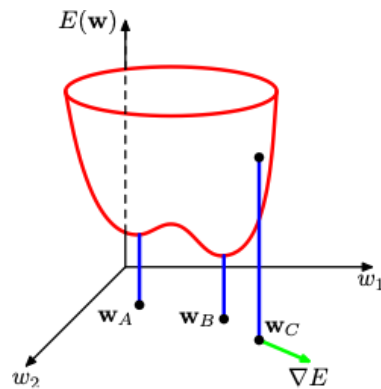
DEEP LEARNING MODEL

1. **Architectures** — MLP, CNN, Transformer
2. **Tasks** — Classification & Segmentation

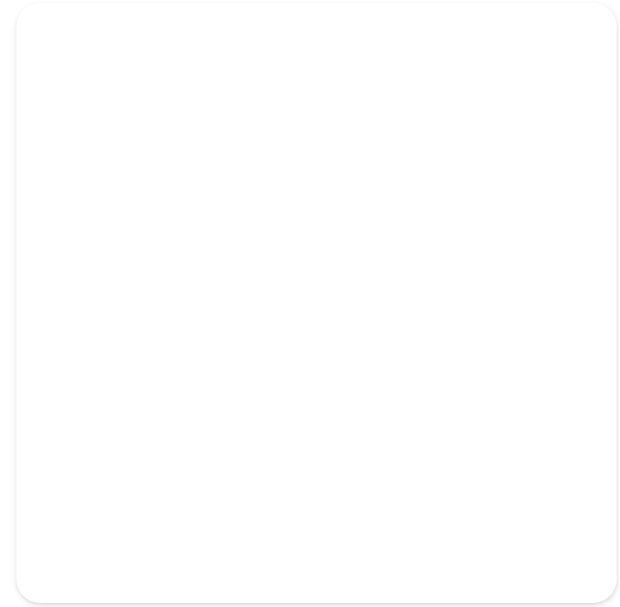


MODEL TRAINING

1. **Loss surfaces** and gradient descent
2. Forward pass → Loss → Backprop → Update

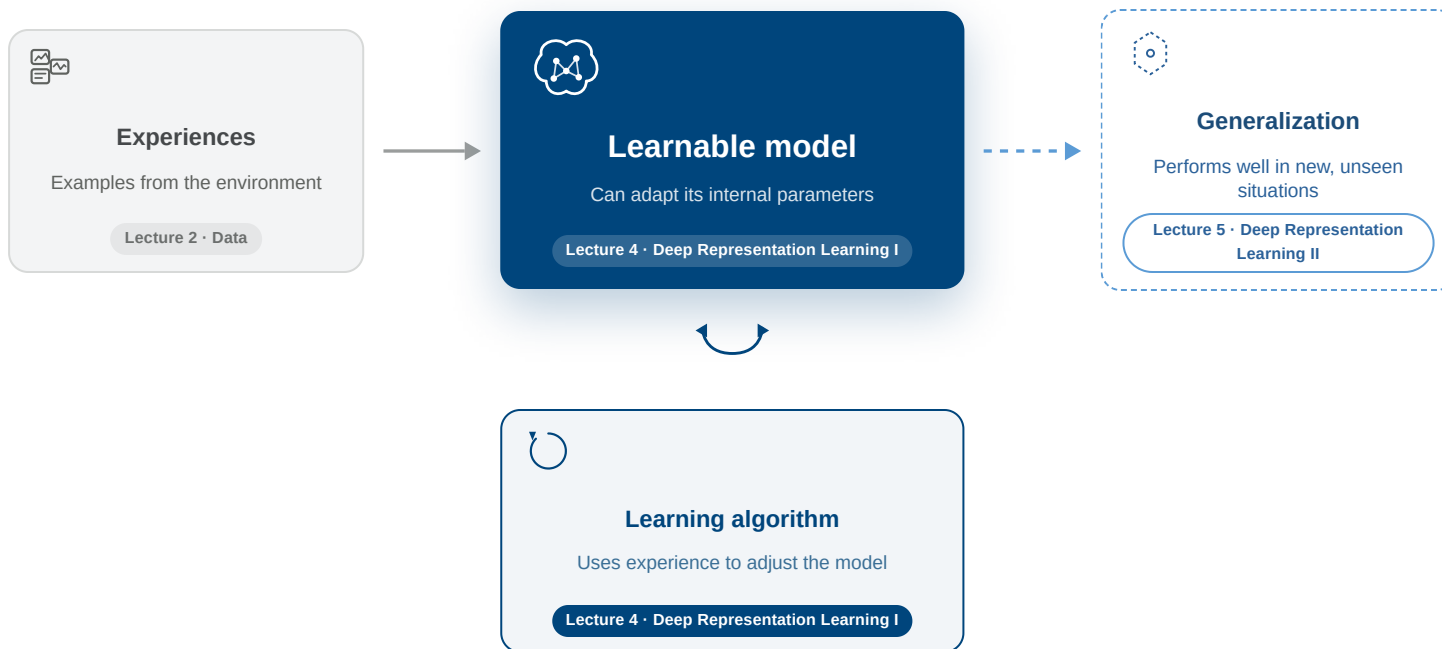


The Aspects of Learning



via [GIPHY](#), original source [imgur.com/gallery/FBMm2DO](#)

What Does a System Need to Learn?



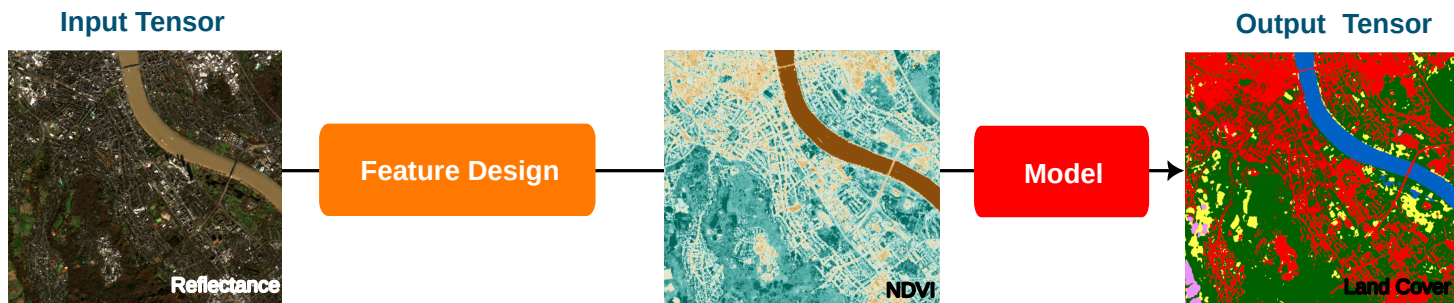
INGREDIENTS FOR LEARNING

SUCCESS CRITERION

Why Learn Representations?

Multilayer Perceptrons Gradient Descent & Backpropagation

Recap: The Classic ML Pipeline



Timeline and Evolution of Deep Learning

<2012	2015 onwards	>2020 →
Feature Design: Which features extract the most information?	Model Design: Which architecture is best for my tensors?	Loss Design: Which loss function or training strategy?
Data: small labelled	Data: large labelled	Data: large unlabelled
Models: Random Forest, Boosting, Lin. Regression	Models: MLPs, CNNs, RNNs	Model: Transformers

Learning the representation.

Multilayer Perceptrons

From Logistic Regression to a Learned Representation

Familiar baseline

$$x \mapsto w^{\text{top}} x + b \mapsto \sigma(\cdot) \mapsto \hat{y}$$

MLP perspective

$$x \mapsto h_{\theta}(x) \mapsto \text{decision layer} \mapsto \hat{y}$$

Separate the representation learner from the final decision layer.

Why Nonlinear Activations?

If every layer were only linear, stacking layers would still be one linear transformation.

Nonlinear activations let the network reshape feature geometry, so classes can become easier to separate with a simple final boundary.

Our Learned Understanding

CHINESE

□□□□□□□□□□□□□□

ARABIC

لقد تعلّم عقلنا أن يفهم هذه الجملة.

HEBREW

המוח שלנו למד להבין את המשפט הזה.

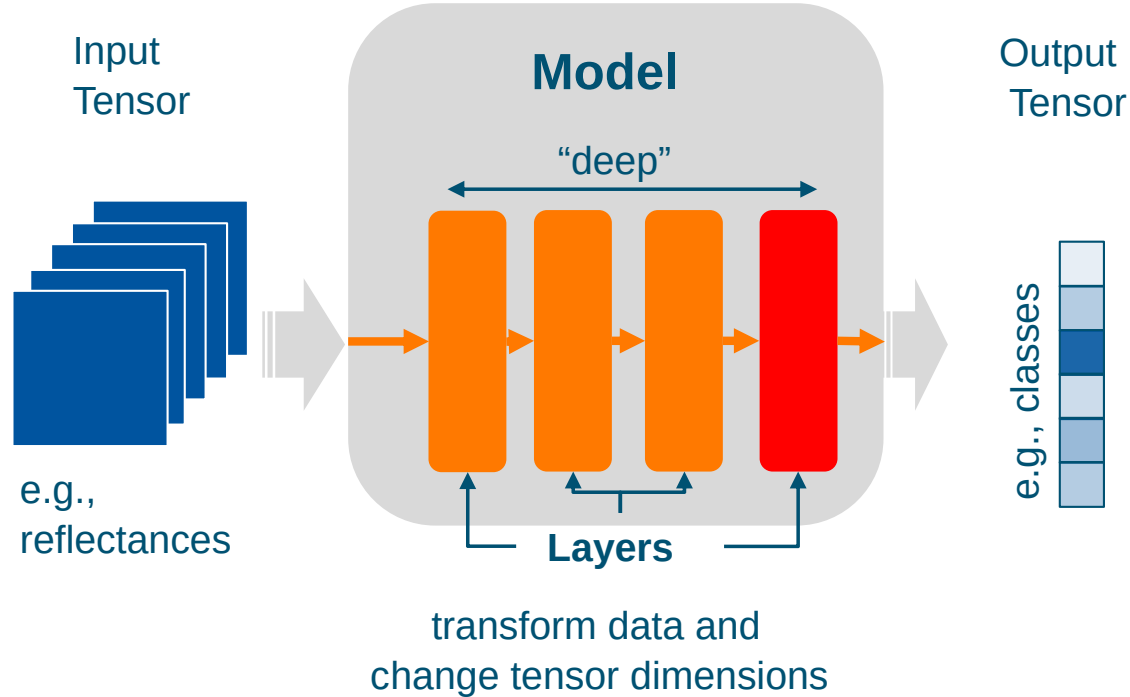
KOREAN

□□□ □□□ □ □□□ □□□□ □□ □□□□□.

LATIN

Our mind has learned to understand this sentence.

Deep Learning Model



Sentinel-2 as an Image Tensor

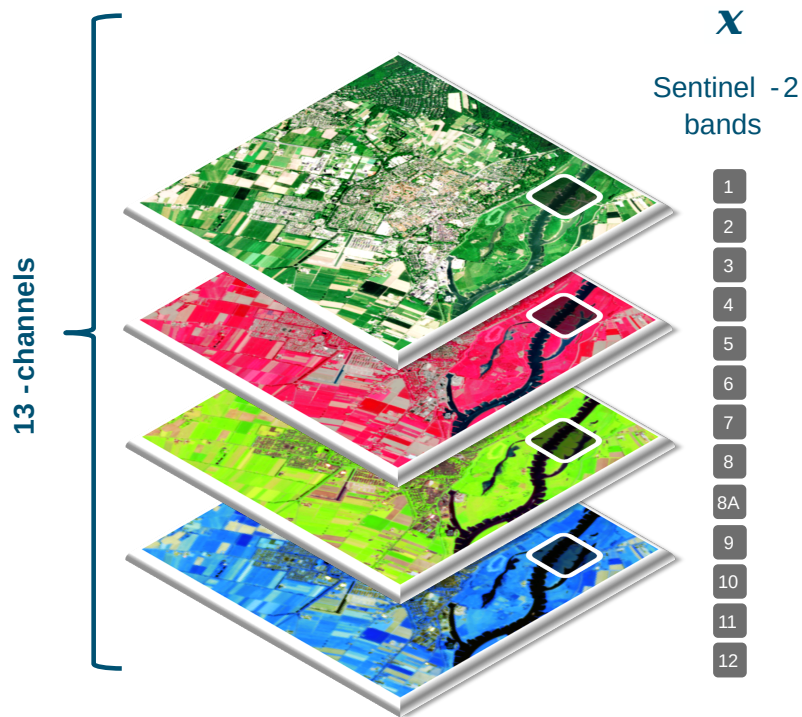


Image Classification

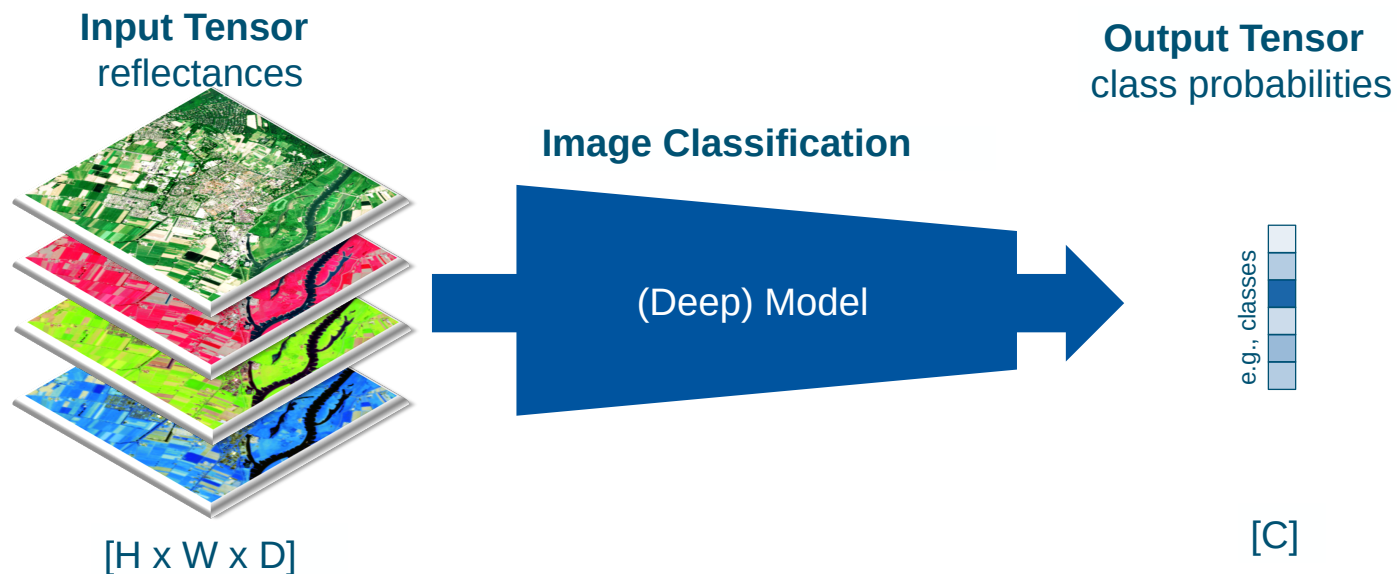
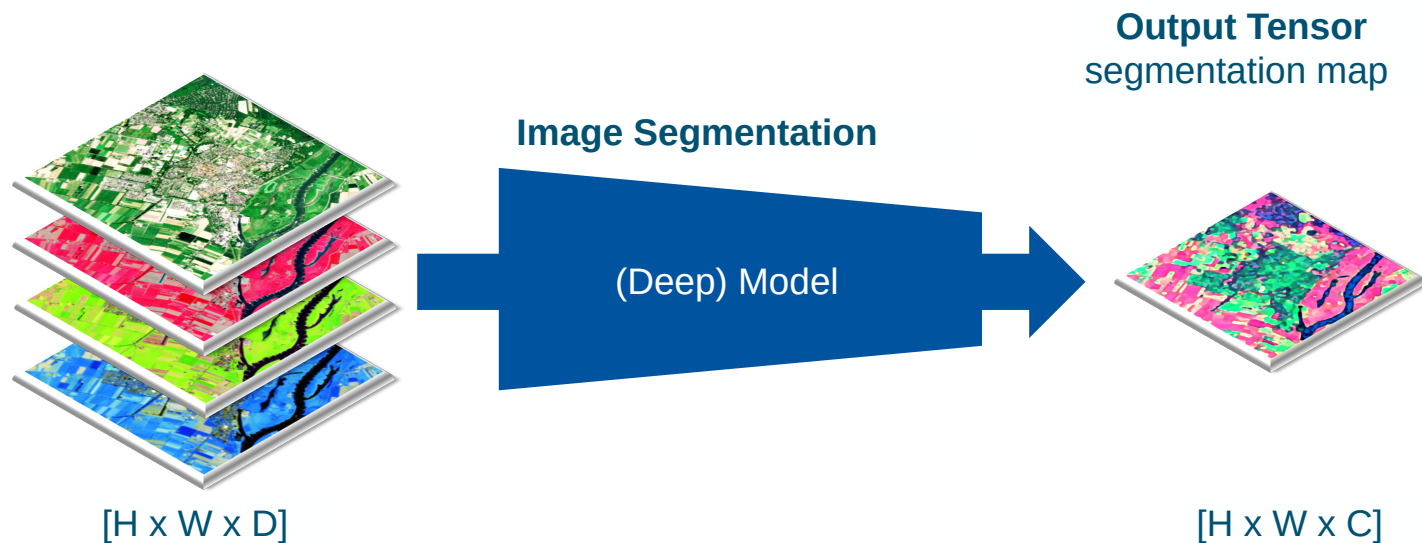


Image Segmentation



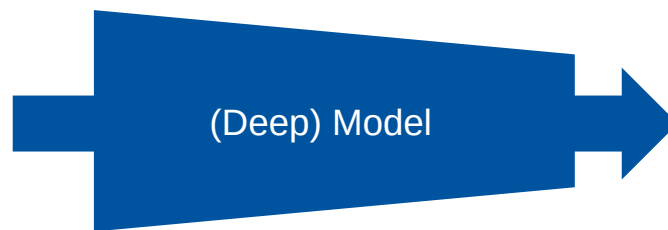
Object Detection

Input Tensor
reflectances



$[H \times W \times 3]$

Object Detection

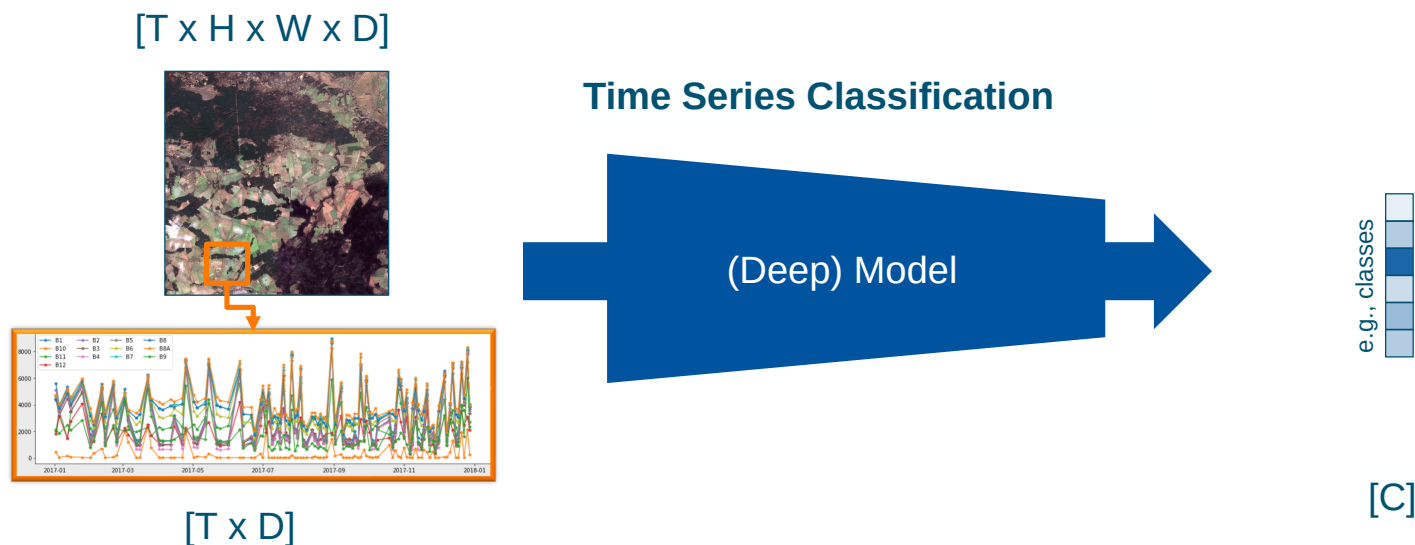


Output Tensor
bounding boxes

X	Y	H	W	
Green	Green	Green	Green	Bbox 1
Red	Red	Red	Red	Bbox 2
Yellow	Yellow	Yellow	Yellow	Bbox 3
Blue	Blue	Blue	Blue	...

$[N \times 4]$

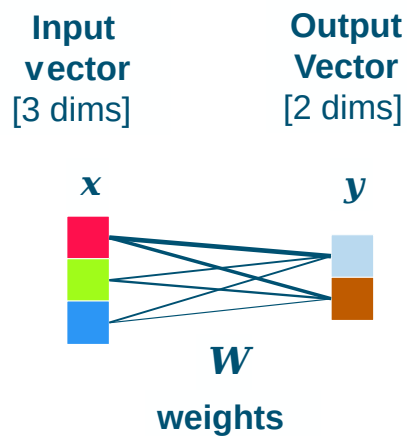
Time Series Classification



Overview: Deep Model Architectures

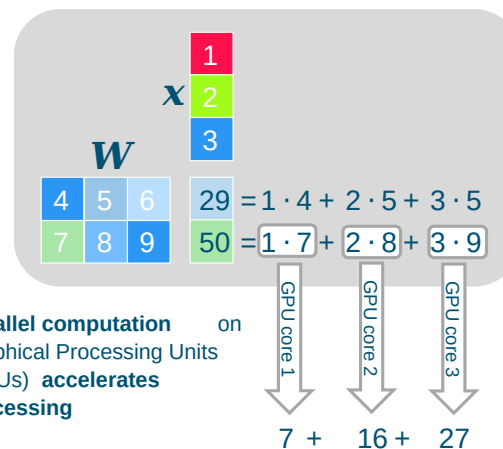
1950s	1990s	2015	2020s →
MLP ▪ Linear Projections ▪ Activation Functions ▪ Linear Algebra & Biological Analogy ▪ Parallelism on GPUs	CNN ▪ Convolutions ▪ Pooling ▪ Recep. Fields	ResNet ▪ Deeper Networks ▪ Batch Normalization ▪ Skip Connections ▪ Classification & Segmentation Tasks	Transformers ▪ Attention Mechanism ▪ Self-Attention (SA) ▪ Multi-Head SA ▪ Transformer Layers

Linear Transformation



Linear Transformation

$$y = Wx = \sum w_i x_i$$

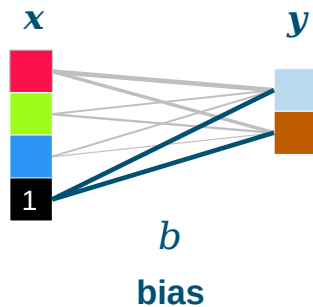


Parallel computation on
Graphical Processing Units
(GPUs) accelerates
processing

Linear Transformation with a Bias Term

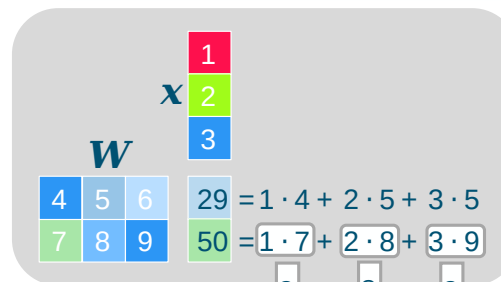
Input
vector
[3 dims]

Output
Vector
[2 dims]

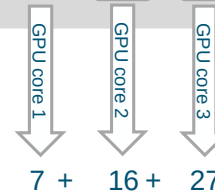


Linear Transformation

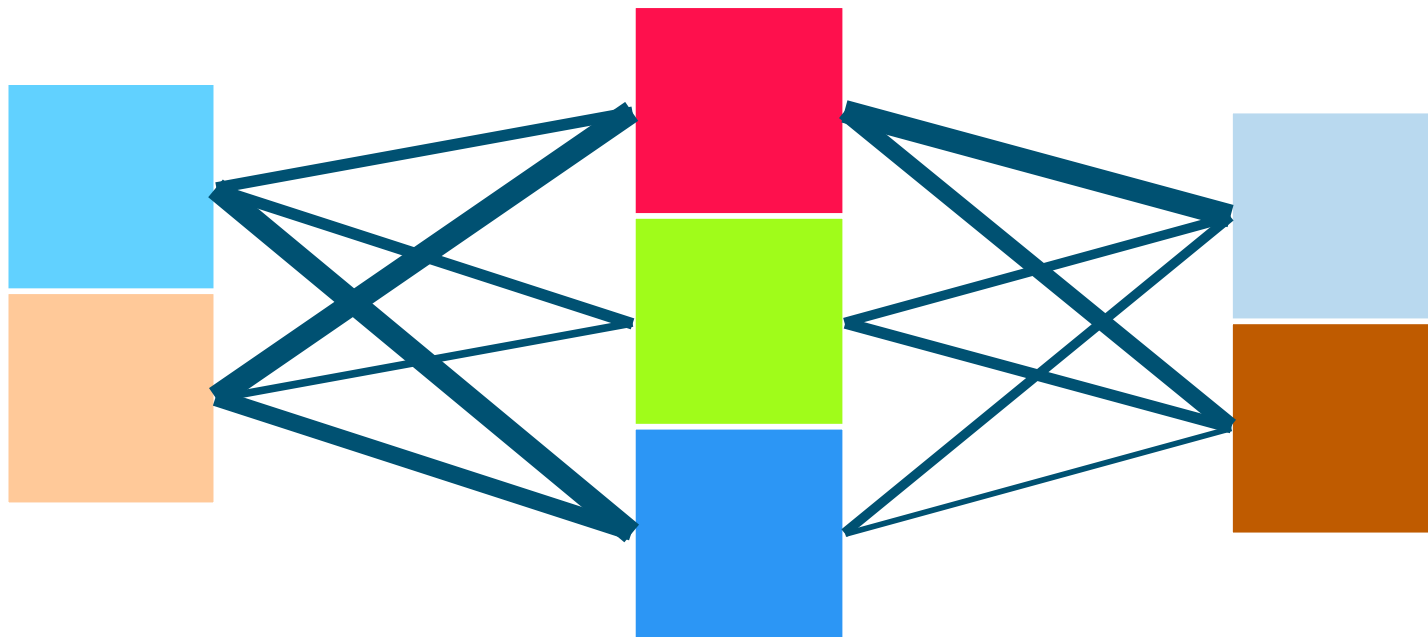
$$y = Wx = \sum w_i x_i + b$$



Parallel computation on
Graphical Processing Units
(GPUs) accelerates
processing

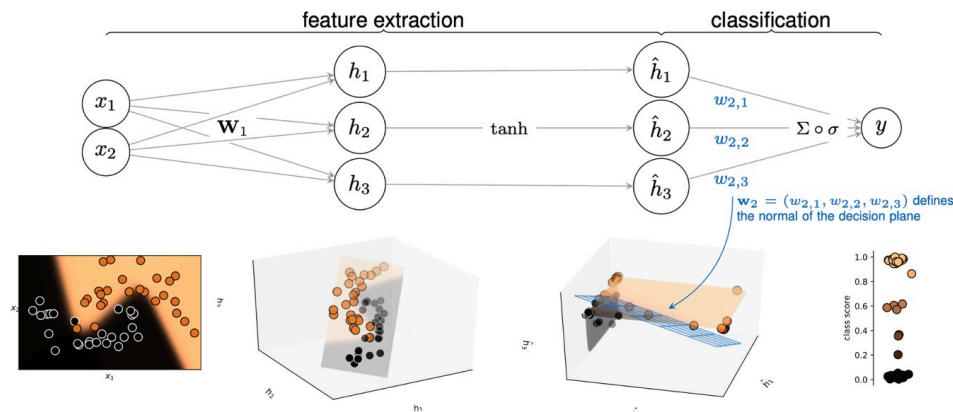


Weighted Connections



More "Deeper" Layers → more Complex Transformations

More "Deeper" Layers → more Complex Transformations



(a) Input space
 $\mathbf{x} = (x_1, x_2)$

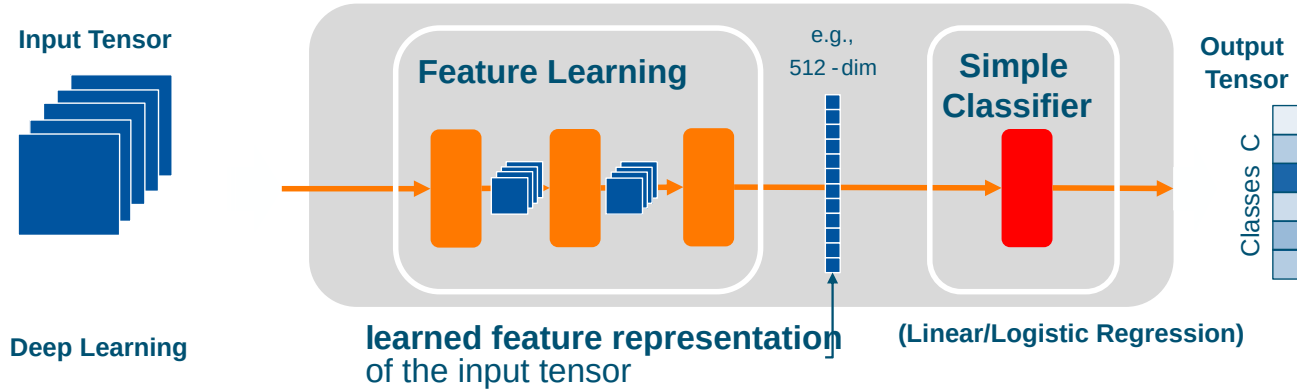
(b) linear projection
 $w_1^T \mathbf{x}$ into $\mathbb{R}^3$

(c) non-linear distortion
with $\tanh(w_1^T \mathbf{x})$

(d) Output class probabilities
 $y \in [0, 1]$

 Loading slide...

Machine Learning and Deep Learning



Machine Learning

Input Tensor



Design knowledge in **designing** features mathematically

Features Design

(e.g., NDVI, NDWI, GLCM Textures)

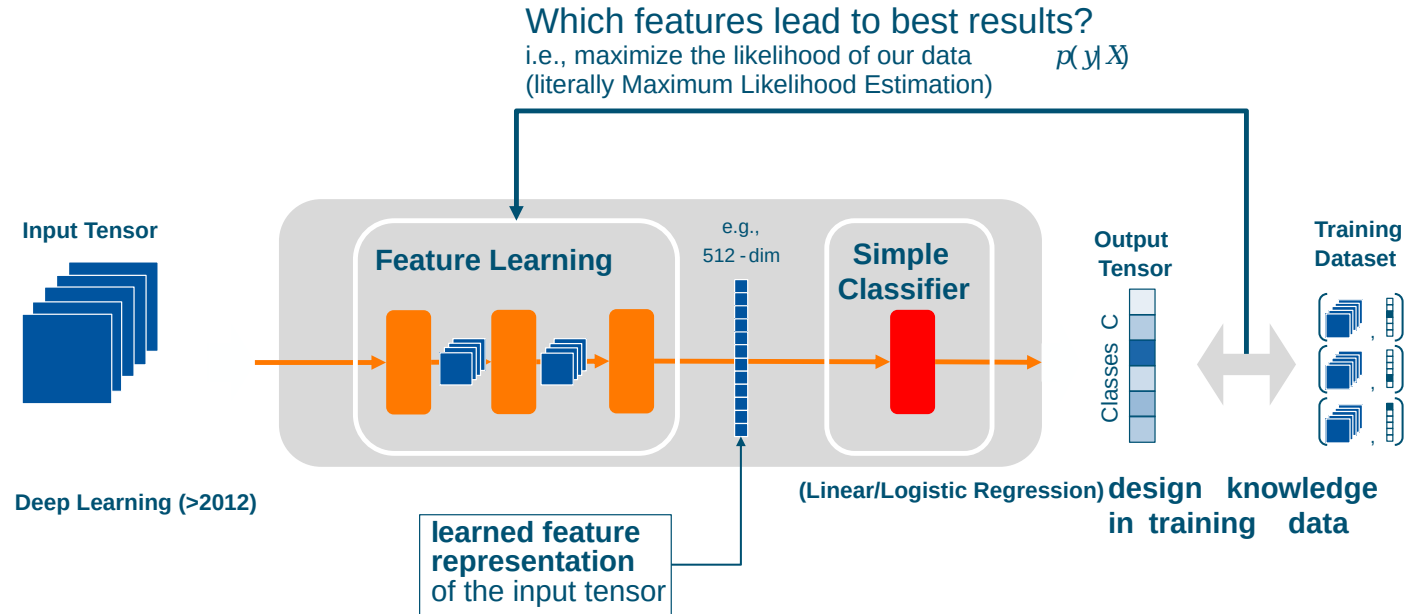
Complex Classifier

(Random Forest, Boosting)

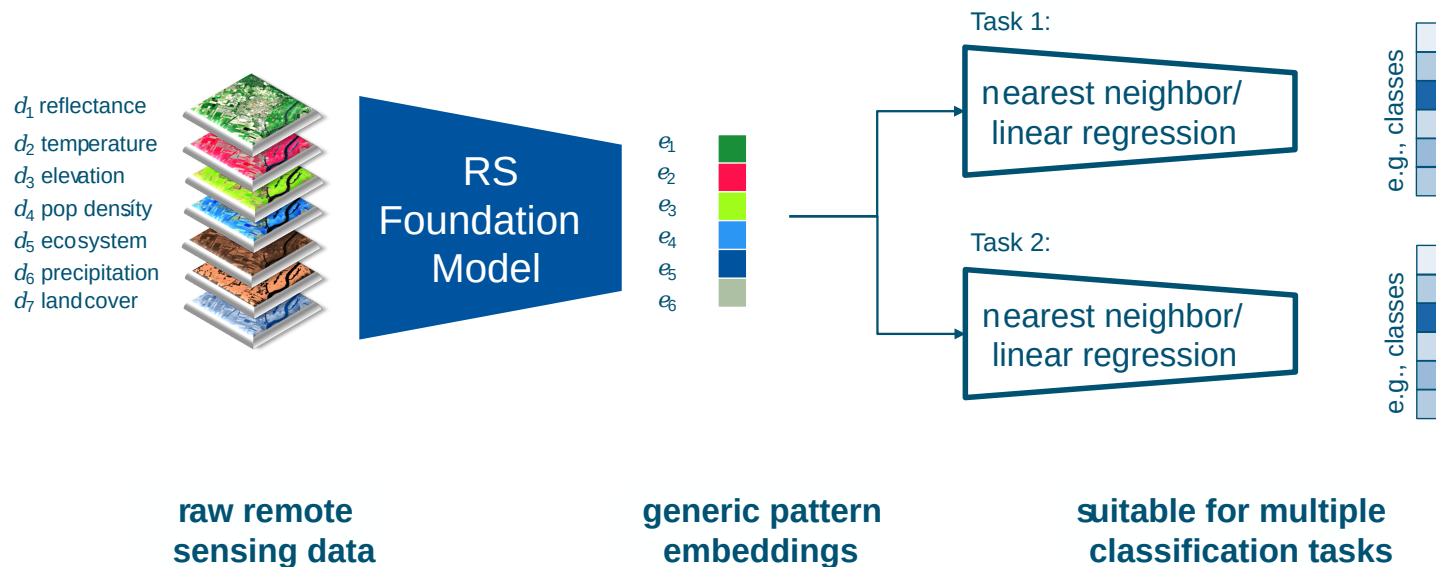
Output Tensor



Feature Learning



Foundation Models



Block 2 Takeaway

The final classifier can stay simple.

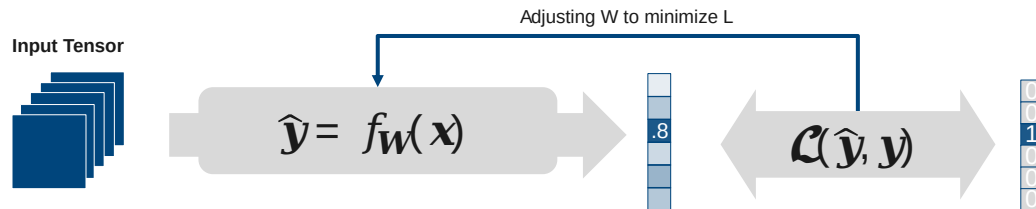
The network learns a representation in which the task becomes simple.

But how are all these parameters learned?

Learning the parameters.

Gradient Descent & Backpropagation

Training: Adjusting Weights to Minimize Loss



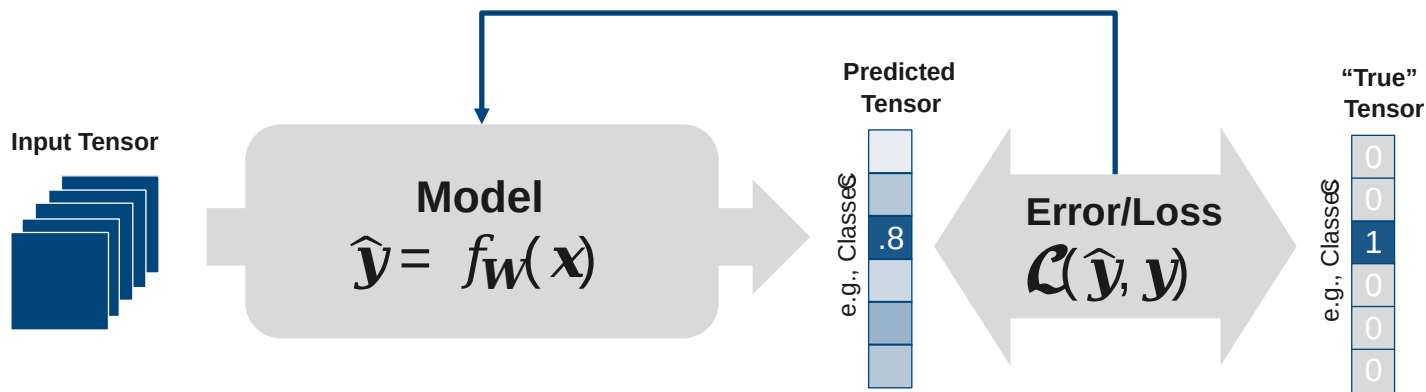
$$\mathcal{L}(\mathcal{D}_{\text{train}}) = \sum_{\mathbf{x}, \mathbf{y} \in \mathcal{D}_{\text{train}}} \mathcal{L}(f(\mathbf{x}), \mathbf{y})$$



The Learning Objective: argmin

Minimize Error/Loss function

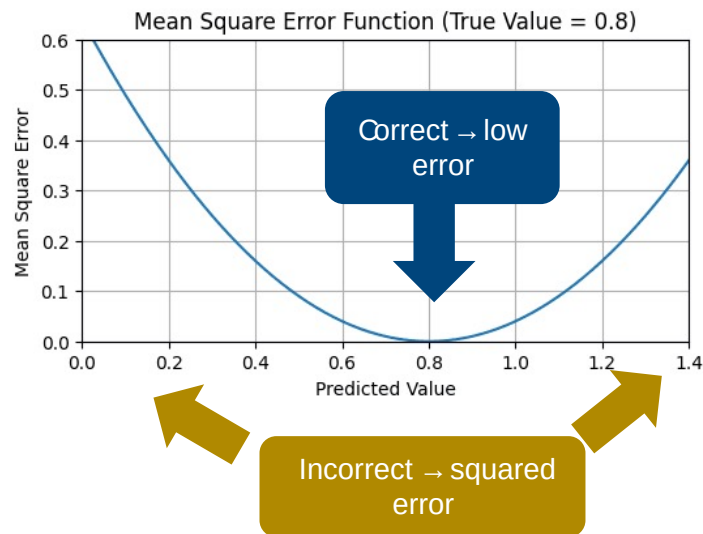
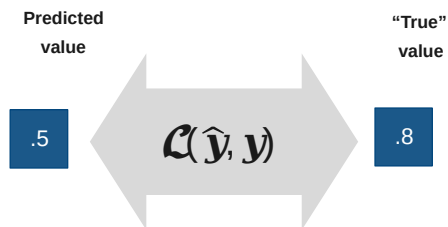
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f(\mathbf{x}), \mathbf{y})$$



Loss Function: Mean Squared Error

Regression: Mean Squared Error

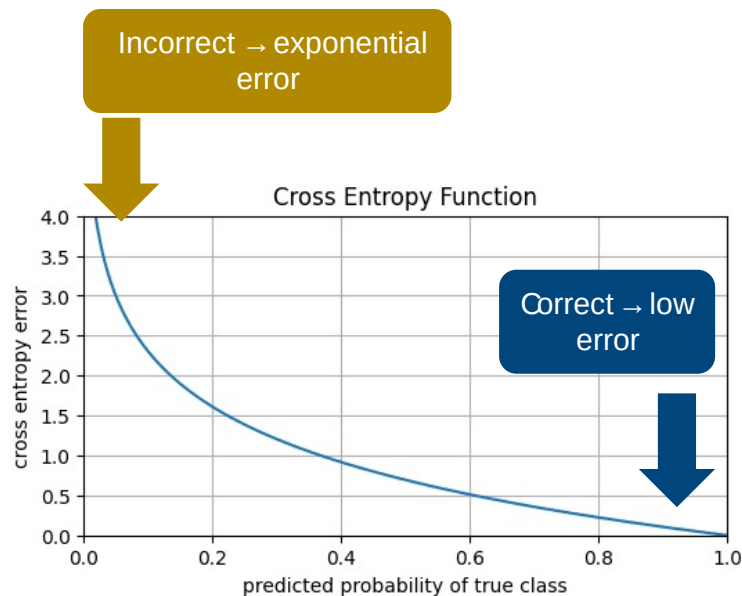
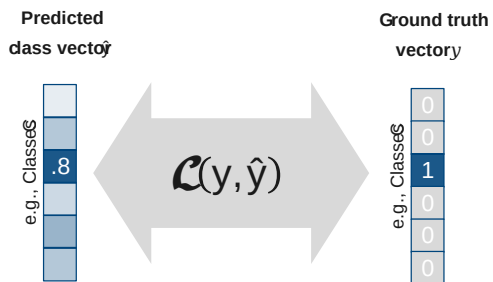
$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$$



Loss Function: Cross-Entropy

Classification: Cross Entropy

$$\mathcal{L}(y, \hat{y}) = \sum_{c \in \text{Classes}} -y_c \log(\hat{y}_c)$$



Loss Surfaces

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(i+1)} = \mathbf{W}^i - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

more complex models have more complex loss surfaces

Gradient Descent

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(i+1)} = \mathbf{W}^i - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

new weights

old weights



Gradients of the Loss function over a “batch” of training data $\mathcal{D}_{\text{train}}$

“What weights change the loss the most?”

Gradient Descent vs Backpropagation

Gradient Descent

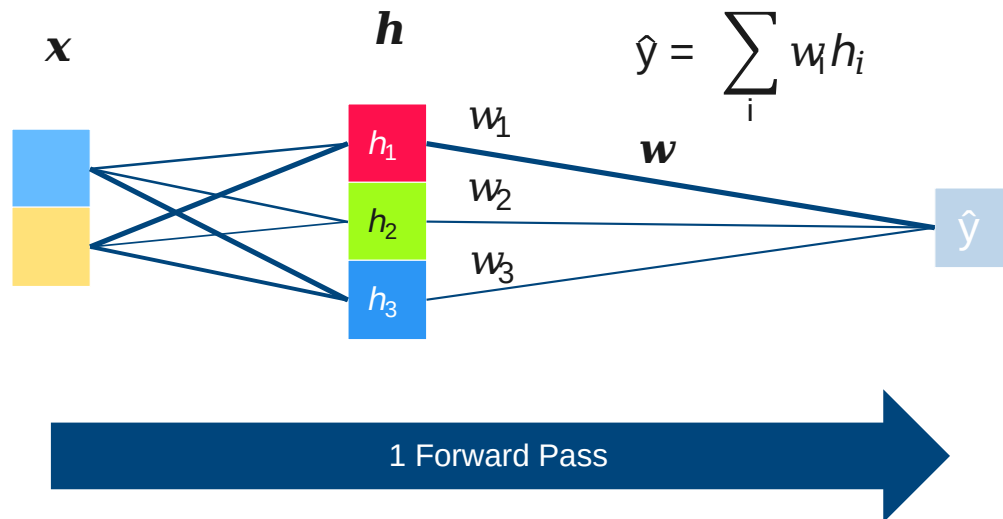
How to update parameters once gradients are known.

Backpropagation

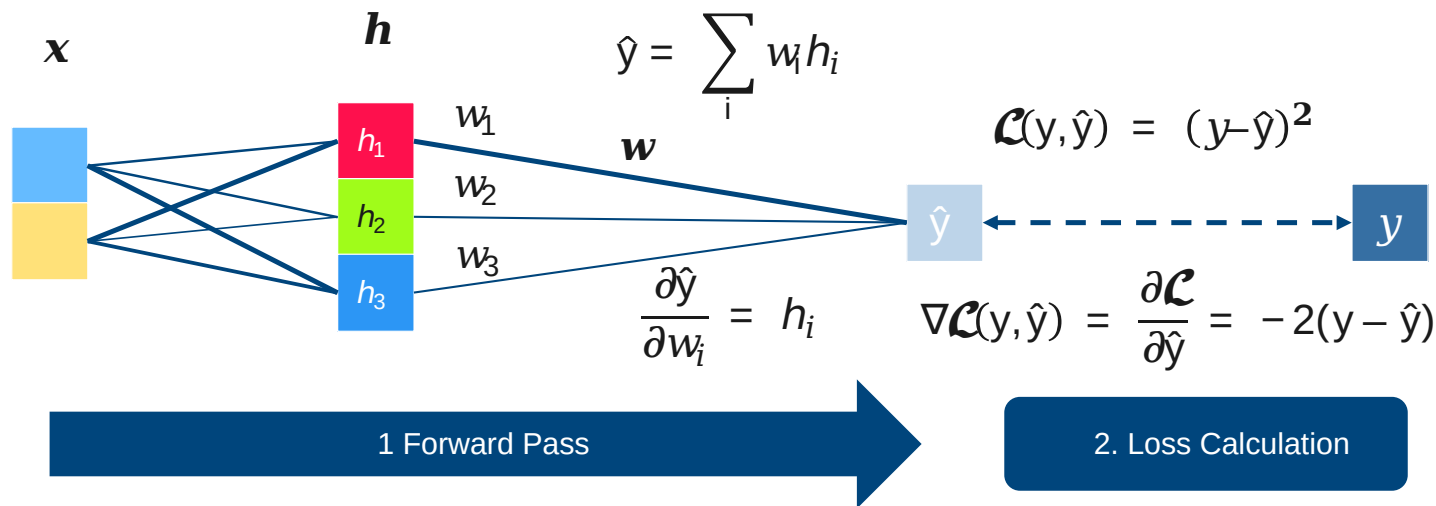
How to compute gradients efficiently through many layers.

Backprop gives the gradients; gradient descent uses them to step the parameters.

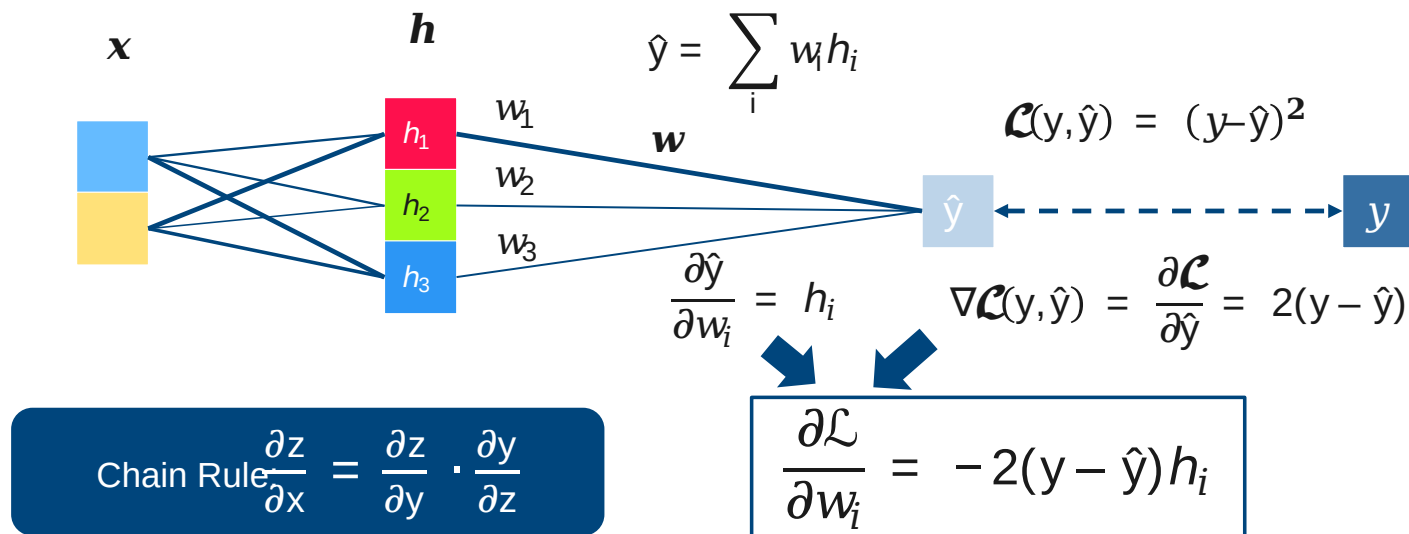
Backpropagation



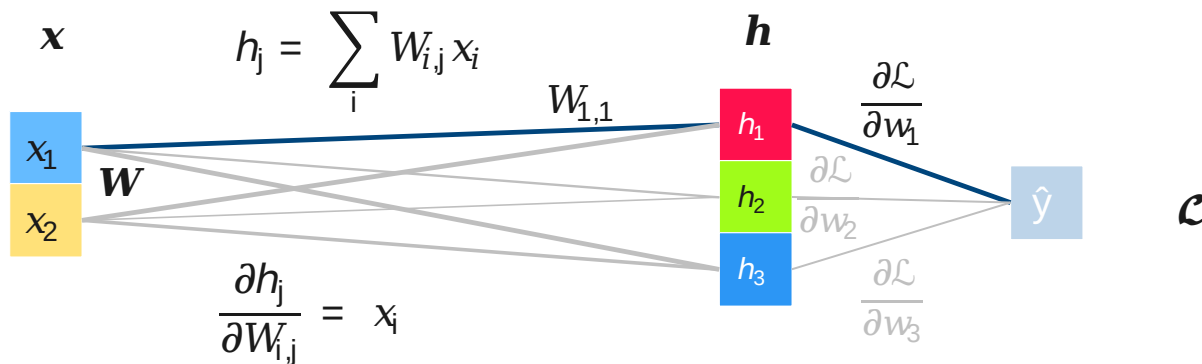
Backpropagation



Backpropagation

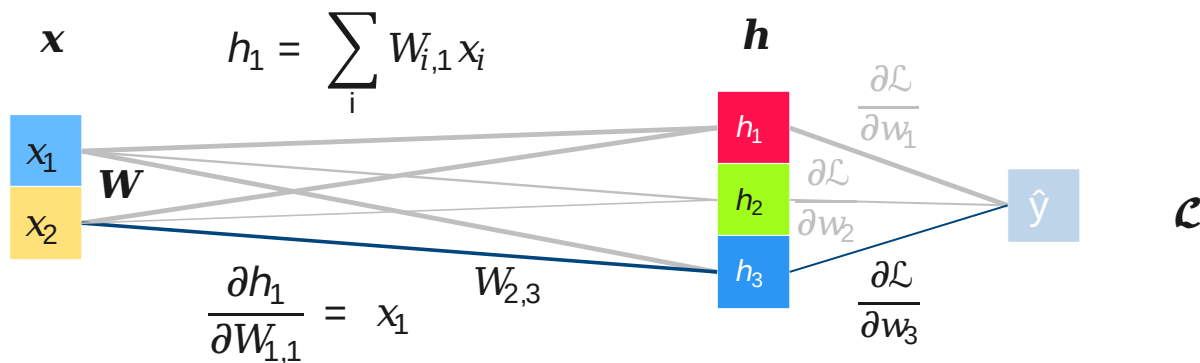


Backpropagation



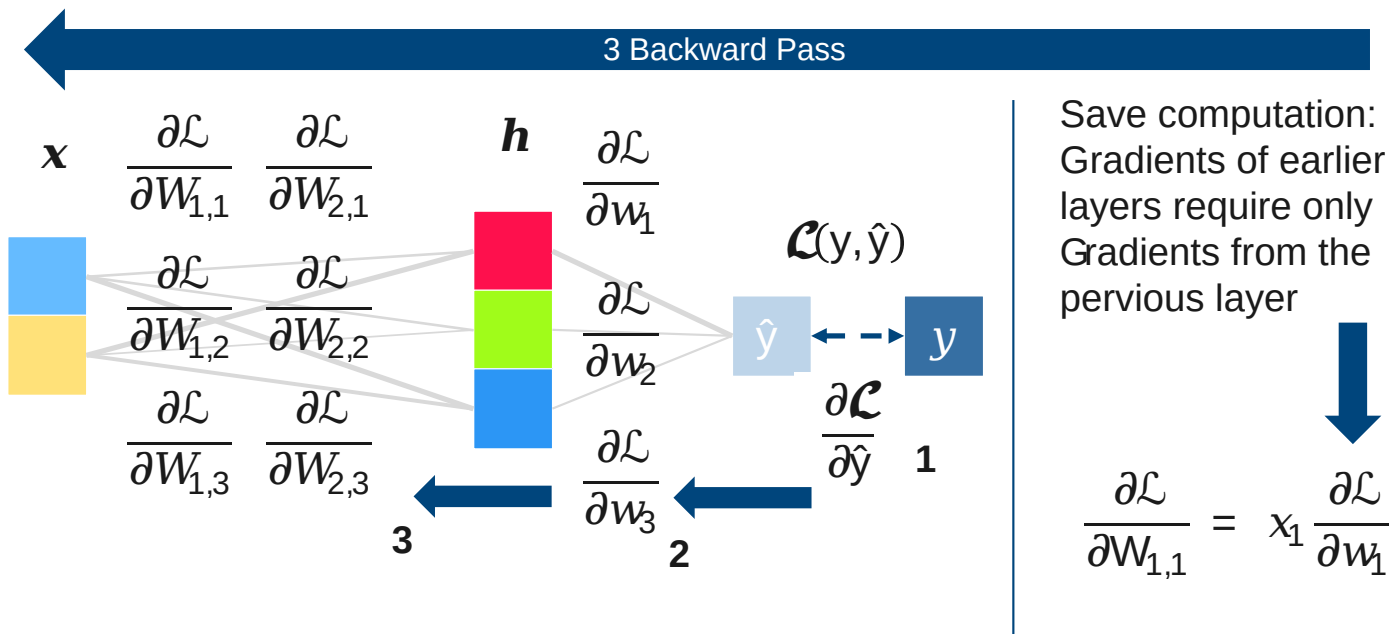
$$\frac{\partial \mathcal{L}}{\partial W_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial h_1} = -x_1 2(y - \hat{y}) h_1$$

Backpropagation

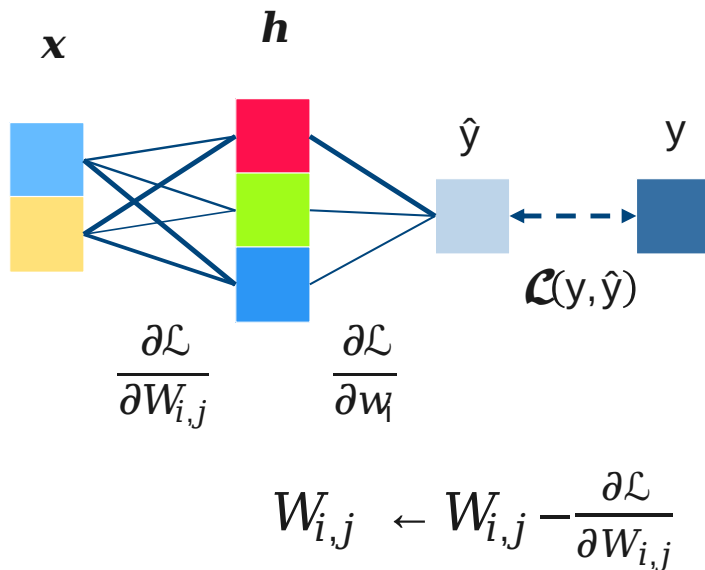


$$\frac{\partial \mathcal{L}}{\partial W_{2,3}} = x_2 \frac{\partial \mathcal{L}}{\partial w_3} = -x_2 2(y - \hat{y}) h_3$$

Backpropagation



Backpropagation



Forward Pass

```
y_pred = model(x)
```

Loss Computation

```
loss = MSE(y_pred, y)
```

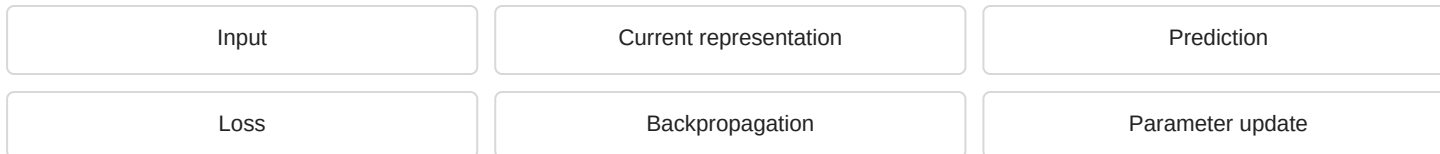
Backward Pass

```
loss.backward()
```

Update Gradients

```
optimizer.step()
```

Close the Representation-Learning Loop



Training changes parameters; changed parameters change the representation.